

Data Structure Lab. Project #2(실습)

2026 년 10 월 16 일

Due date: 2026 년 11 월 6 일 금요일 23:59:59 까지

본 프로젝트에서는 B+ Tree, AVL Tree 자료 구조를 이용하여 중고거래 마켓 상품 검색 시스템을 구현한다. 중고거래 마켓 상품 검색 시스템은 중고거래 플랫폼에 등록된 상품 정보를 저장하고, 상품 ID 와 가격을 기준으로 효율적으로 검색할 수 있는 시스템이다.

상품 정보는 [상품 ID, 판매자 ID, 카테고리, 가격, 상태]로 구성되어 있다. B+ Tree 는 가격을 기준으로 판매 중인 상품을 관리하며, 특정 가격대의 상품을 검색하는 데 사용한다. AVL Tree 는 상품 ID 를 기준으로 등록된 모든 상품을 관리하며, 특정 상품을 빠르게 찾는 데 사용한다.

같은 가격의 상품이 여러 개 존재할 수 있으므로 B+ Tree 는 중복 키를 처리해야 한다. 상품이 판매 완료되면 B+ Tree 에서 삭제하고, AVL Tree 에는 상태만 갱신한다. 두 트리는 동일한 상품 데이터를 서로 다른 키로 가리킨다. 자료구조의 구축 방법 및 조건은 Program Implementation 에서 자세히 설명한다.

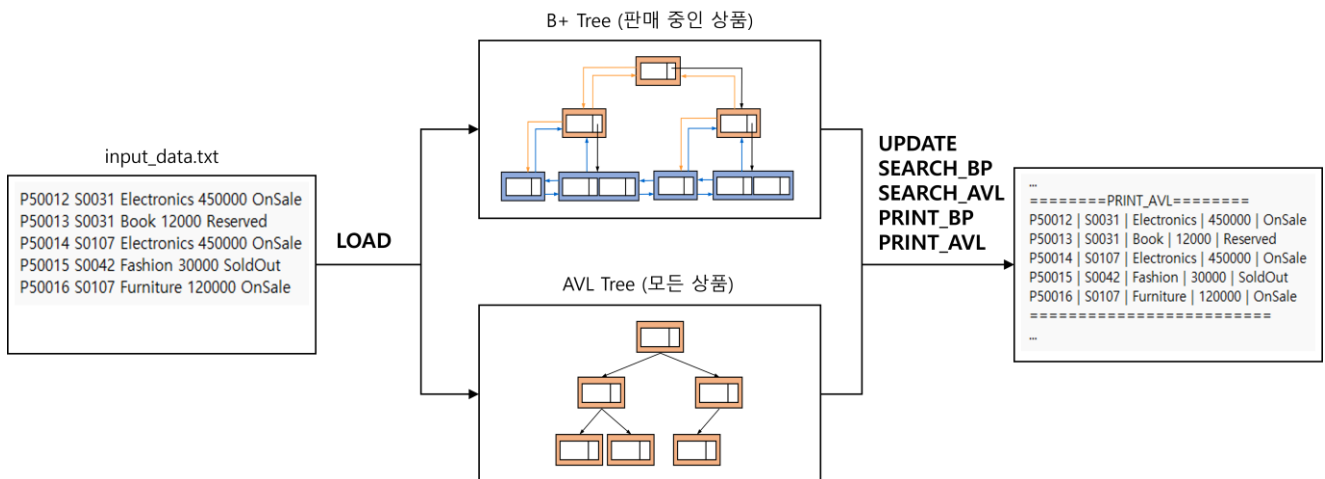


그림 1. 중고거래 마켓 상품 검색 시스템 구조

□ Program implementation

1) ItemData

상품 ID, 판매자 ID, 카테고리, 가격, 상태 정보가 포함된 데이터셋 input_data.txt 를 LOAD 명령어로 읽어 '상품 정보 클래스(ItemData)'에 저장한다. 데이터는 ' '(스페이스바 한 번)으로 구분한다.

상품 ID

1. 상품 ID 는 항상 6 자리(대문자 'P'+숫자 5 자리)로 구성되어 있다.
2. 상품 ID 는 항상 고유하며, 중복인 경우는 없다고 가정한다.

판매자 ID

1. 판매자 ID 는 항상 5 자리(대문자 'S' + 숫자 4 자리)로 구성되어 있다.
2. 한 판매자가 여러 상품을 등록할 수 있으므로 판매자 ID 는 중복될 수 있다.

카테고리

1. 카테고리는 5 개 (Electronics, Fashion, Furniture, Book, Sports)로 구성되어 있다.

가격

1. 가격은 1 이상의 정수이다.
2. 서로 다른 상품이 같은 가격을 가질 수 있다.

상태 정보

1. 상태 정보는 3 개 (OnSale, Reserved, SoldOut)로 구성되어 있다
2. 상태가 "OnSale" 또는 "Reserved"인 상품은 판매 중인 상품으로, B+ Tree 와 AVL Tree 에 모두 저장한다.
3. 상태가 "SoldOut"인 상품은 판매 완료 상품으로, AVL Tree 에만 저장한다.
4. 상태 정보는 UPDATE 명령어로 변경할 수 있으며, "SoldOut"은 더 이상 변경할 수 없다.

2) B+ Tree

1. B+ Tree 의 차수는 3 으로 구현한다.
2. B+ Tree 는 가격을 기준으로 구성하며, 판매 중("OnSale", "Reserved")인 상품만 저장한다.
3. Index Node(BpTreeIndexNode)와 Data Node(BpTreeDataNode)로 구성되며, 각 Node Class 는 B+ Tree Node Class(BpTreeNode)를 상속받는다.
4. Data Node 는 `map<int, vector<ItemData*>>` 형태로 데이터를 가진다. 키는 가격이며, 같은 가격의 상품은 해당 키의 vector 에 삽입 순서대로 저장한다.
5. 같은 가격의 상품이 추가되면 새로운 키를 만들지 않고 기존 키의 vector 에 추가한다. 노드의 split 은 vector 의 크기와 관계없이 노드 안의 키 개수를 기준으로 판단한다.
6. UPDATE 명령어로 상품이 "SoldOut"이 되면 해당 상품을 vector 에서 제거한다. vector 가 비면 해당 가격 키를 B+ Tree 에서 삭제하며, 이때 발생하는 underflow 는 노드의 상태에 따라 재분배(redistribution) 또는 병합(merge)으로 처리한다.
7. B+ Tree 는 ItemData 객체를 포인터로만 참조하며, 제거 시 메모리를 해제하지 않는다.

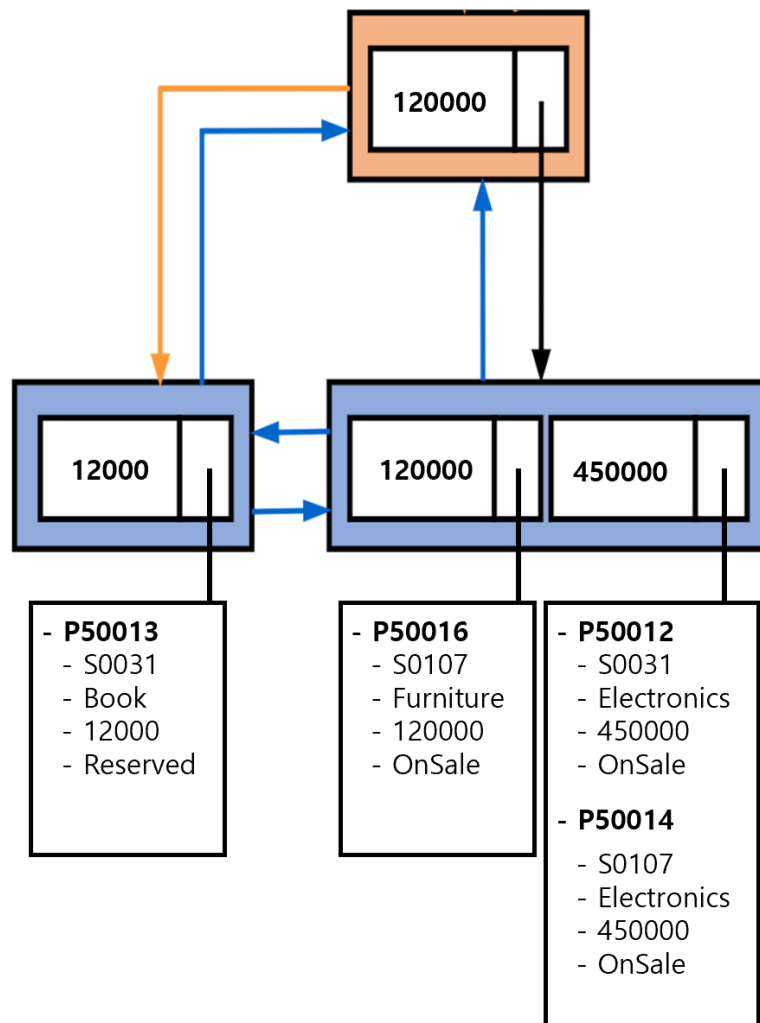


그림 2. B+ Tree 구축 예시

3) AVL Tree

1. AVL Tree 는 상태와 관계없이 등록된 모든 상품을 저장한다.
2. AVL Tree 는 상품 ID 를 기준으로 정렬하며, 대소문자를 구별하지 않는다.
3. AVL Tree 는 AVL Tree Class(AVLTree)와 AVL Tree Node Class(AVLNode)로 구성된다.
AVLNode 는 ItemData 포인터, 왼쪽/오른쪽 자식 포인터, Balance Factor 를 멤버로 가진다.
4. AVL Tree 는 각 Node 마다 Balance Factor 를 가지고 있으며, 이를 활용해 Tree 의 균형을 유지한다.
5. 상품의 상태가 변경되면 AVL Tree 에 저장된 ItemData 의 상태 정보를 갱신한다. AVL Tree 에서 노드를 삭제하는 경우는 없다.
6. ItemData 의 메모리는 AVL Tree 가 소유하며, 프로그램 종료 시 AVL Tree 에서 해제한다.

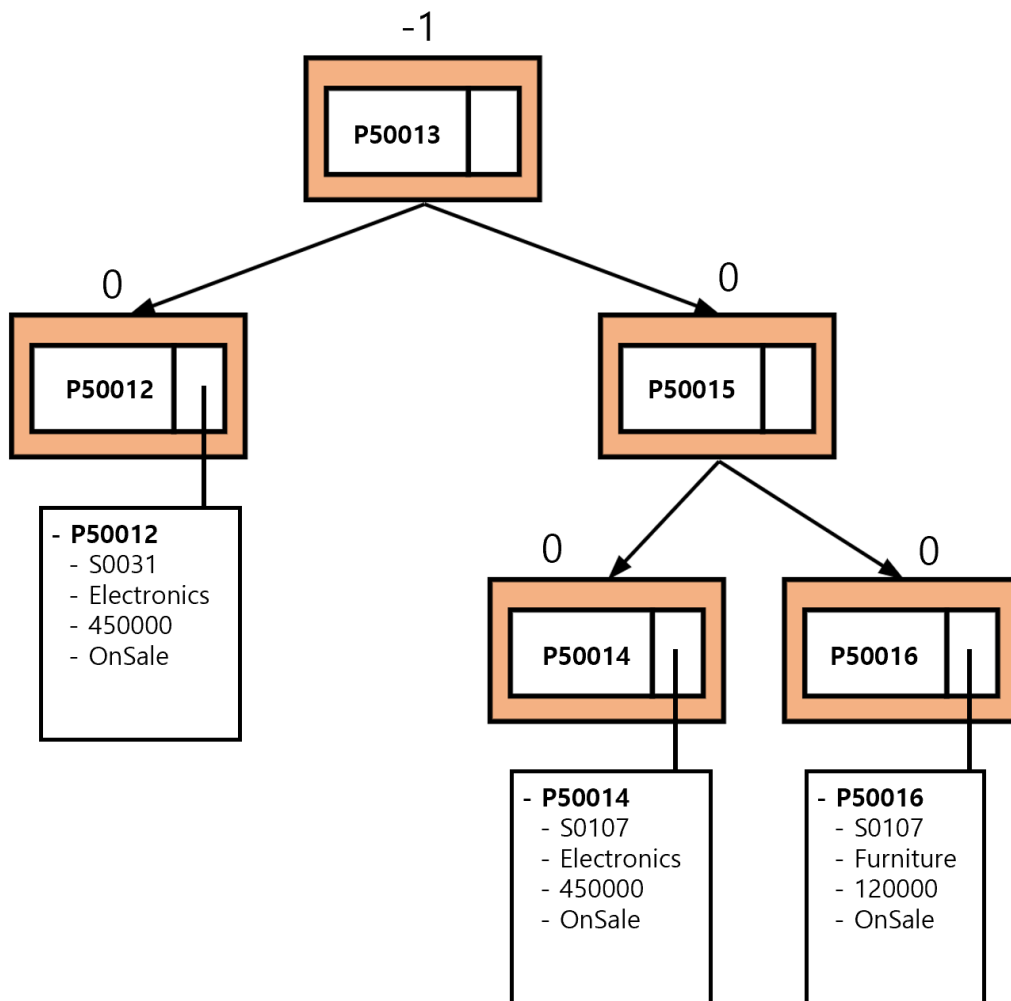


그림 3. AVL Tree 구축 예시

□ Functional Requirements

- 출력 포맷은 포맷에 대한 예시일 뿐 실제 출력되는 데이터들과는 차이가 있을 수 있습니다.

명령어	명령어 사용 예 및 기능
LOAD	사용 예) LOAD 텍스트 파일의 데이터 정보를 불러오는 명령어로, 텍스트 파일에 데이터 정보가 존재할 경우 텍스트 파일을 읽어 모든 데이터를 AVL Tree 에 저장하고, 상태가 "OnSale" 또는 "Reserved"인 상품은 B+ Tree 에도 저장한다. [에러 코드 출력] • 텍스트 파일이 존재하지 않는 경우 • 텍스트 파일 안에 데이터가 존재하지 않는 경우 • B+ Tree 또는 AVL Tree 가 이미 존재하는 경우 [출력 예시] <pre>=====LOAD===== Success ===== =====ERROR===== 100 =====</pre>
ADD	사용 예) ADD P50020 S0031 Sports 80000 새 상품을 등록하는 명령어로, 상품 ID, 판매자 ID, 카테고리, 가격 순서로 4 개의 인자를 입력한다. 추가된 상품의 상태는 "OnSale"로 고정하며, AVL Tree 와 B+ Tree 에 모두 저장한다. B+ Tree 에 같은 가격의 상품이 이미 존재하면 해당 키의 vector 맨 뒤에 추가한다. [에러 코드 출력] • 인자 4 개를 입력하지 않은 경우 • 이미 존재하는 상품 ID 인 경우 • 상품 ID, 판매자 ID 의 형식이 맞지 않는 경우 • 카테고리가 정의된 5 개 중 하나가 아닌 경우 • 가격이 1 이상의 정수가 아닌 경우 [출력 예시] <pre>=====ADD===== P50020 S0031 Sports 80000 OnSale</pre>

	<pre>===== =====ERROR===== 200 =====</pre>
UPDATE	사용 예) UPDATE P50012 SoldOut 상품의 상태를 변경하는 명령어로, 상품 ID 와 변경할 상태를 인자로 입력한다. AVL Tree 에서 상품을 찾아 상태를 갱신하며, 상태가 "SoldOut"으로 변경되면 해당 상품을 B+ Tree 에서 삭제한다. 허용되는 상태 변경은 다음 4 가지이다. 1. OnSale → Reserved 2. Reserved → OnSale 3. OnSale → SoldOut 4. Reserved → SoldOut [에러 코드 출력] • 인자 2 개를 입력하지 않은 경우 • AVL Tree 에 해당 상품이 없는 경우 • 상태가 정의된 3 개 중 하나가 아닌 경우 • 위 4 가지를 제외한 모든 상태 변경 (같은 상태로의 변경, "SoldOut"에서의 변경 포함) [출력 예시] <pre>=====UPDATE===== P50012 S0031 Electronics 450000 SoldOut ===== =====ERROR===== 300 =====</pre>
SEARCH_BP	사용 예 1) SEARCH_BP 450000 사용 예 2) SEARCH_BP 10000 200000 B+ Tree 에 저장된 판매 중인 상품을 가격으로 검색하는 명령어로, 1 개 또는 2 개의 인자를 입력한다. 인자가 1 개이면 해당 가격의 상품을 모두 출력한다. 인자가 2 개(최저 가격, 최고 가격)이면 최저 가격 이상, 최고 가격 이하인 상품을 Data Node 의 연결을 따라 가격 오름차순으로 출력한다. 같은 가격의 상품은 삽입 순서대로 출력한다. [에러 코드 출력]

	• 인자를 1 개 또는 2 개 입력하지 않은 경우 • 인자가 1 이상의 정수가 아닌 경우 • 최저 가격이 최고 가격보다 큰 경우 • B+ Tree 에 데이터가 없는 경우 • 해당하는 상품이 없는 경우 [출력 예시] 1) SEARCH_BP 450000 =====SEARCH_BP===== P50012 S0031 Electronics 450000 OnSale P50014 S0107 Electronics 450000 OnSale ===== 2) SEARCH_BP 10000 200000 =====SEARCH_BP===== P50013 S0031 Book 12000 Reserved P50016 S0107 Furniture 120000 OnSale ===== =====ERROR===== 400 =====
SEARCH_AVL	사용 예) SEARCH_AVL P50015 AVL Tree 에서 상품 ID 로 상품을 검색하는 명령어로, 판매 완료된 상품도 검색할 수 있다. [에러 코드 출력] • 인자 1 개를 입력하지 않은 경우 • AVL Tree 에 데이터가 없는 경우 • 해당하는 상품이 없는 경우 [출력 예시] =====SEARCH_AVL===== P50015 S0042 Fashion 30000 SoldOut ===== =====ERROR===== 500 =====

PRINT_BP	사용 예) PRINT_BP B+ Tree 에 저장된 모든 상품을 가격 오름차순으로 출력한다. 가장 왼쪽 Data Node 부터 연결을 따라 순회하며, 같은 가격의 상품은 삽입 순서대로 출력한다. [에러 코드 출력] B+ Tree 에 저장된 데이터가 없는 경우 [출력 예시] <pre> =====PRINT_BP===== P50013 S0031 Book 12000 Reserved P50016 S0107 Furniture 120000 OnSale P50012 S0031 Electronics 450000 OnSale P50014 S0107 Electronics 450000 OnSale ===== =====ERROR===== 600 ===== </pre>
PRINT_AVL	사용 예) PRINT_AVL AVL Tree 에 저장된 모든 상품을 상품 ID 오름차순으로 출력한다. 중위 순회(in-order)로 출력한다. [에러 코드 출력] AVL Tree 에 저장된 데이터가 없는 경우 [출력 예시] <pre> =====PRINT_AVL===== P50012 S0031 Electronics 450000 OnSale P50013 S0031 Book 12000 Reserved P50014 S0107 Electronics 450000 OnSale P50015 S0042 Fashion 30000 SoldOut P50016 S0107 Furniture 120000 OnSale ===== =====ERROR===== 700 ===== </pre>
EXIT	사용 예) EXIT

	프로그램상의 메모리를 해제하며, 프로그램을 종료한다. 메모리 누수 시 감점 요인이다. [출력 예시] =====EXIT===== Success =====
--	---

□ 동작 별 에러 코드

동작	에러 코드
LOAD	100
ADD	200
UPDATE	300
SEARCH_BP	400
SEARCH_AVL	500
PRINT_BP	600
PRINT_AVL	700
잘못된 명령어	1000

□ Requirements in implementation

- 모든 명령어는 command.txt 에 저장하여 순차적으로 읽고 처리한다.
- 모든 명령어는 반드시 대문자로 입력한다. (인자는 상관없음)
- 명령어에 인자(Parameter)가 모자라거나 필요 이상으로 입력 받으면 에러 코드를 출력한다.
- 예외처리에 대해 반드시 에러 코드를 출력한다.
- 메모리 누수 발생 시, 감점 10%
- 잘못된 출력 시, 기능 별 감점
- 출력은 "출력 포맷"을 반드시 따라 한다.
- log.txt 파일에 출력 결과를 반드시 저장한다.
 - log.txt 가 이미 존재할 경우 텍스트 파일 가장 뒤에 이어서 추가로 저장한다.

□ STL 사용 규정

- STL 은 vector, stack, queue, string 만 사용할 수 있으며, map 은 BpTreeDataNode 의 데이터 저장 용도로만 사용한다.
- 상품 데이터를 B+ Tree, AVL Tree 외부의 별도 컨테이너에 저장하여 검색 또는 출력에 사용하는 경우 해당 명령어는 0 점 처리한다.

□ 구현 시 반드시 정의해야 하는 Class

- ItemData: 상품 정보 Class
- AVLTree: AVL Tree Class
- AVLNode: AVL Tree 의 Node Class
- BpTree: B+ Tree Class
- BpTreeNode: B+ Tree 의 Node Class
- BpTreeIndexNode: B+ Tree 의 Index Node Class
- BpTreeDataNode: B+ Tree 의 Data Node Class
- Manager: 다른 Class 들의 동작을 관리해 프로그램을 조정하는 역할 수행

□ Files

- input_data.txt: 프로그램에 추가할 상품 정보가 저장된 파일
- command.txt: 프로그램을 동작시키는 명령어들을 저장하고 있는 파일
- log.txt: 프로그램 출력 결과를 모두 저장하고 있는 파일

□ 제한사항 및 구현 시 유의사항

- 반드시 제공되는 코드(github 주소 참고)를 이용하여 구현하며 작성된 소스 파일의 이름과 클래스와 함수 이름 및 형태를 임의로 변경하지 않는다.
- 함수의 반환형과 매개변수는 구현에 맞게 조정할 수 있으며, 클래스의 함수 및 변수는 자유롭게 추가 구현이 가능하다.
- 제시된 Class 를 각 기능에 알맞게 모두 사용한다.
- 프로그램 구조에 대한 디자인이 최대한 간결하도록 고려하여 설계한다.
- 채점 시 코드를 수정해야 하는 일이 없도록 한다.
- 주석은 반드시 영어로 작성한다. (한글로 작성하거나 없으면 감점)
- 프로그램은 반드시 리눅스(Ubuntu 20.04)에서 동작해야 한다. (컴파일 에러 발생 시 감점)
 - 제공되는 Makefile 을 사용하여 테스트하도록 한다.
- 인터넷에서 공유된 코드, AI 로 생성된 코드나 다른 학생이 작성한 코드를 절대 카피하지 않도록 하며, 적발 시 해당 프로젝트 0 점 처리됨을 명시한다.

□ 채점 기준

1. 코드

채점 기준	점수
LOAD 명령어가 정상 동작하는가?	1
ADD 명령어가 정상 동작하는가?	1
UPDATE 명령어가 정상 동작하는가?	2
SEARCH_BP 명령어가 정상 동작하는가?	2
SEARCH_AVL 명령어가 정상 동작하는가?	2
PRINT_BP 명령어가 정상 동작하는가?	1
PRINT_AVL 명령어가 정상 동작하는가?	1
총합	10

- 채점 기준 이외에도 조건 미달 시 감점 (linux 컴파일 에러, 파일 입출력 X, 주석 미흡 등)

□ 제출기한 및 제출방법

2. 제출기한

- 2026 년 11 월 6 일 일요일 23:59:59 까지 클래스(KLAS)에 제출
- (추가 제출: 2026 년 11 월 7 일 월요일 00:59:59 까지, 10% 감점)

3. 제출방법

- 소스코드를 압축하여 제출
- 확장자가 .cpp, .h, .txt 가 아닌 파일은 제출하지 않음 (제출시 감점)

4. 제출 형식

- 학번_DS_project2.tar.gz (ex. 2026XXXXXX_DS_project2.tar.gz)
- 제출 형식을 지키지 않은 경우 감점